# CHƯƠNG III. NÉN CHỈ MỤC

Truy xuất thông tin

ThS. Phạm Đức Cường
cuongpd@ptit.edu.vn

Posts and Telecommunications
Institute of Technology

Học viện Công nghệ Bưu chính Viễn thông

# Nội dung

1. Lý do cần nén chỉ mục
2. Các quy luật thống kê tập dữ liệu
3. Kỹ thuật nén từ điển
4. Kỹ thuật nén postings

# Vì sao cần nén?

- Sử dụng **ít dung lượng lưu trữ (disk space)** hơn
  - Tiết kiệm một phần chi phí; đồng thời cung cấp cho người dùng **nhiều không gian lưu trữ hơn**
- Có thể lưu trữ **nhiều dữ liệu hơn trong bộ nhớ**
  - Tăng tốc độ xử lý
- Tăng tốc độ truyền dữ liệu từ **đĩa vào bộ nhớ**
  - Tổ hợp thao tác **[đọc dữ liệu đã nén | giải nén]** thường **nhanh hơn** so với **[đọc dữ liệu chưa nén]**

# Vì sao cần nén đối với chỉ mục ngược?

- **Từ điển**
  - Làm cho từ điển đủ nhỏ để có thể **lưu hoàn toàn trong bộ nhớ chính**
  - Làm cho nó đủ nhỏ để có thể lưu thêm **một số danh sách posting** trong bộ nhớ chính
- **Tập tin posting**
  - Giảm dung lượng lưu trữ cần thiết trên đĩa
  - Giảm thời gian cần thiết để đọc các danh sách posting từ đĩa
  - Các **search engine quy mô lớn** lưu trữ một phần đáng kể các posting trong bộ nhớ
    - Nén dữ liệu cho phép lưu được **nhiều posting hơn trong bộ nhớ**
- Chúng ta sẽ xây dựng các **phương pháp nén chuyên biệt cho IR**

# Nhắc lại bộ dữ liệu Reuters RCV1

| Ký hiệu | Thống kê | Giá trị (khoảng) |
|---|---|---|
| **N** | Số lượng tài liệu | **800.000** |
| **L** | Số lượng token trung bình mỗi tài liệu | **200** |
| **M** | Số lượng thuật ngữ (từ phân biệt – word types) | **400.000** |
| | Số byte trung bình cho mỗi token (bao gồm khoảng trắng/dấu câu) | **6** |
| | Số byte trung bình cho mỗi token (không gồm khoảng trắng/dấu câu) | **4,5** |
| | Số byte trung bình cho mỗi thuật ngữ | **7,5** |
| | Số lượng posting **không có vị trí** (non-positional postings) | **100.000.000** |

# Ảnh hưởng của tiền xử lý

| size of | word types (terms) | | | non-positional postings | | | positional postings | | |
|---|---|---|---|---|---|---|---|---|---|
| | dictionary | | | non-positional index | | | positional index | | |
| | Size (K) | Δ% | cumul % | Size (K) | Δ % | cumul % | Size (K) | Δ % | cumul % |
| Unfiltered | 484 | | | 109,971 | | | 197,879 | | |
| No numbers | 474 | -2 | -2 | 100,680 | -8 | -8 | 179,158 | -9 | -9 |
| Case folding | 392 | -17 | -19 | 96,969 | -3 | -12 | 179,158 | 0 | -9 |
| 30 stopwords | 391 | -0 | -19 | 83,390 | -14 | -24 | 121,858 | -31 | -38 |
| 150 stopwords | 391 | -0 | -19 | 67,002 | -30 | -39 | 94,517 | -47 | -52 |
| stemming | 322 | -17 | -33 | 63,812 | -4 | -42 | 94,517 | 0 | -52 |

Câu hỏi: Vì sao một số ô bằng 0 lại tương ứng với những thay đổi (Δ) lớn ở các cột khác?

# Nén không mất thông tin vs. Nén mất thông tin

- **Nén không mất thông tin**
  - Tất cả thông tin đều được bảo toàn
  - Đây là phương pháp **chủ yếu sử dụng trong IR**
- **Nén mất thông tin**
  - Loại bỏ một phần thông tin
- Một số bước tiền xử lý có thể được xem là dạng nén mất thông tin
  - Chuyển về chữ thường
  - Loại bỏ stop words
  - Stemming
  - Loại bỏ số

# Kích thước từ vựng vs. kích thước bộ sưu tập

- **Kích thước từ vựng lớn đến mức nào?**
  - Tức là có bao nhiêu từ phân biệt
- Chúng ta có thể giả định một cận trên không?
  - **Không.** Chỉ riêng các từ có độ dài 20 ký tự:
    - Nếu mỗi ký tự có 70 khả năng khác nhau
    - Thì có ít nhất $70^{20} \approx 10^{37}$ từ khác nhau
- Trong thực tế, khi kích thước bộ sưu tập tăng lên, từ vựng cũng tiếp tục tăng
  - Đặc biệt với Unicode ☺

→ Không gian khả năng còn lớn hơn rất nhiều.

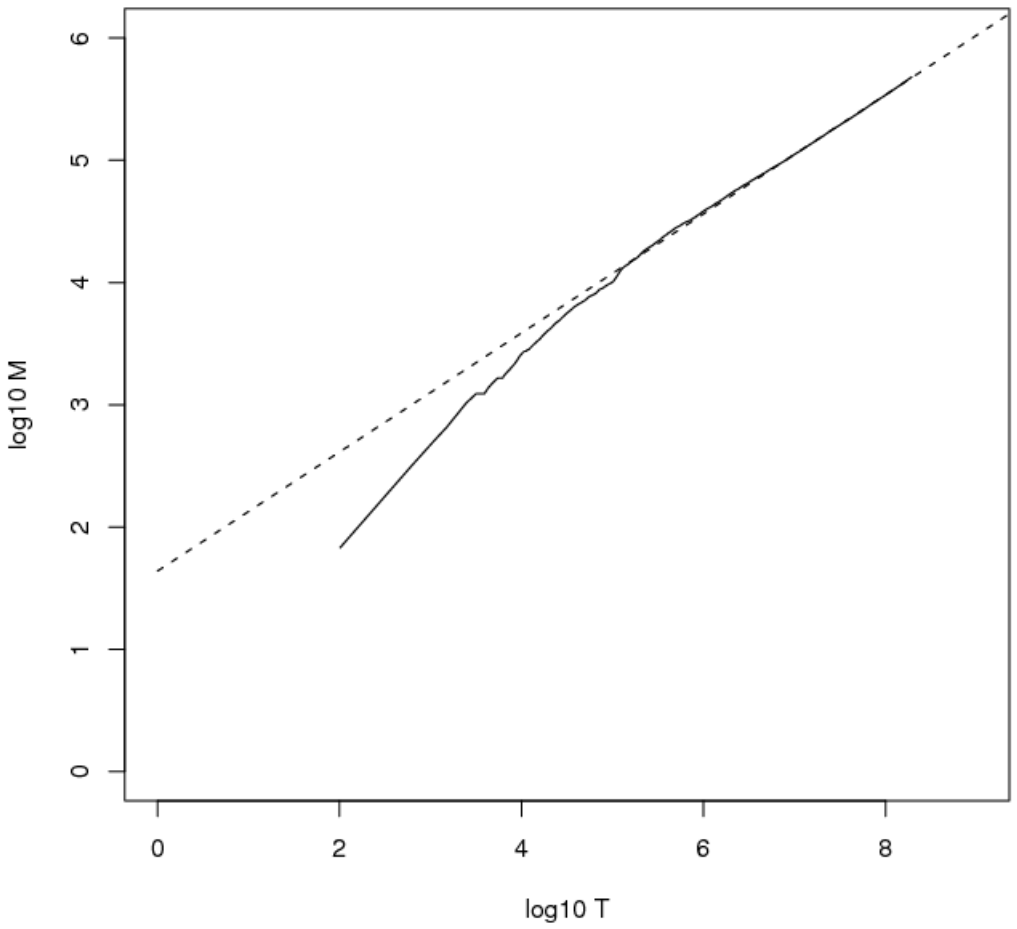
# Định luật Heaps (Heaps' Law)

- $M = kT^b$
- Đối với tập Reuters RCV1, đường nét đứt: $\log_{10}M = 0.49 \log_{10}T + 1.64$ là đường hồi quy bình phương tối thiểu tốt nhất. Suy ra, $M = 10^{1.64}T^{0.49}$, do đó: $k = 10^{1.64} \approx 44$ và $b = 0.49$.

→ Phù hợp thực nghiệm rất tốt đối với Reuters RCV1!

- Đối với 1.000.020 token đầu tiên:
  - Theo định luật, số term dự đoán là: 38.323 terms
  - Thực tế: 38.365 terms

→ Sai lệch rất nhỏ, cho thấy định luật Heaps mô hình hóa rất tốt sự tăng trưởng của kích thước từ vựng



# Bài tập

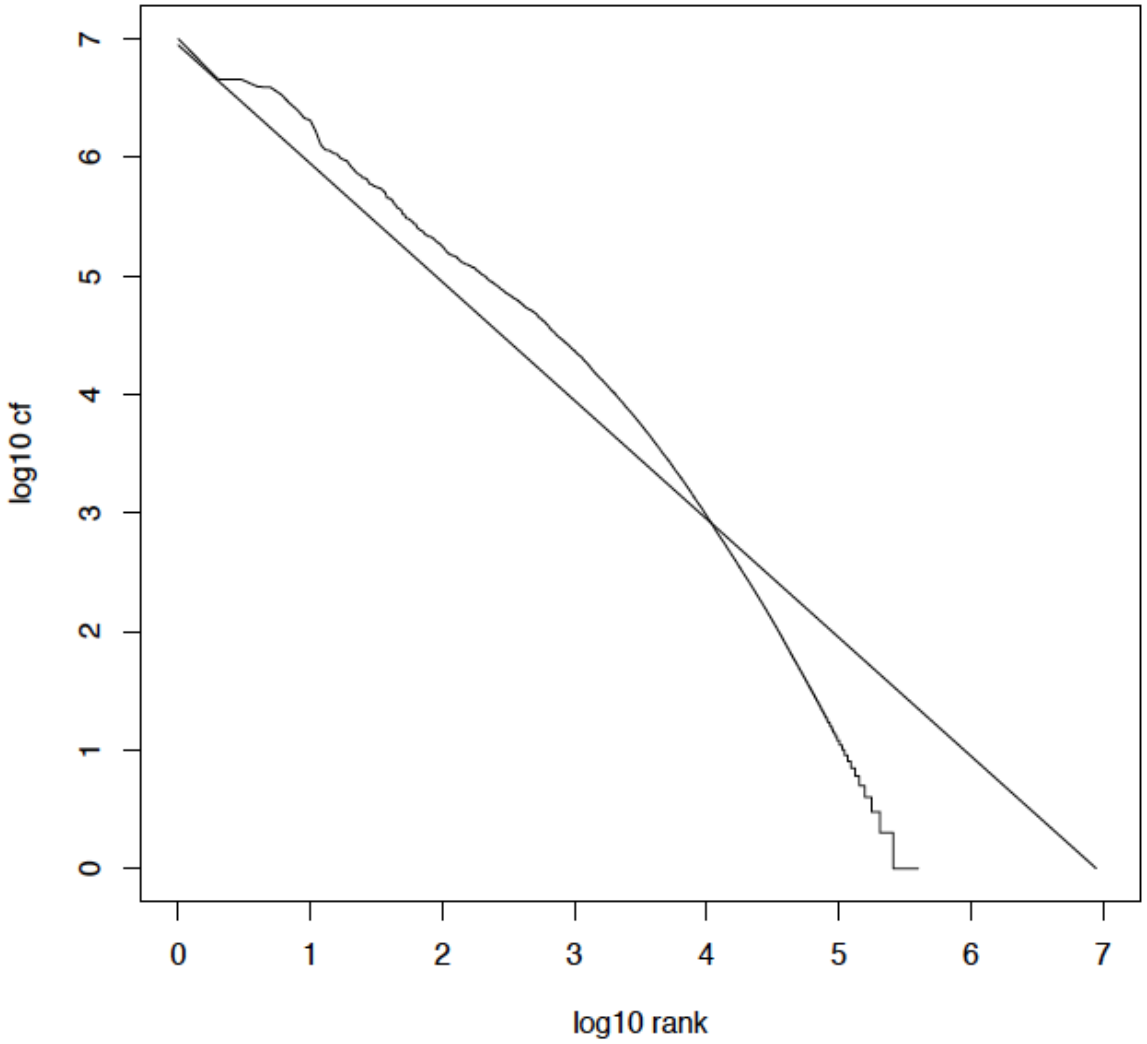
- Việc **bao gồm các lỗi chính tả** sẽ ảnh hưởng thế nào đến Heaps' Law, so với trường hợp **tự động sửa lỗi chính tả**?
- Tính **kích thước từ vựng M** trong trường hợp:
  - Xét một bộ sưu tập trang web:
    - **10.000 token** đầu tiên có **3.000 terms** khác nhau
    - **1.000.000 token** đầu tiên có **30.000 terms** khác nhau
  - Giả sử một công cụ tìm kiếm lập chỉ mục:
    - Tổng cộng **20.000.000.000 trang ($2 \times 10^{10}$)**
    - Trung bình mỗi trang có **200 token**
  - **Kích thước từ vựng M** của toàn bộ bộ sưu tập, theo dự đoán của Heaps' Law, là bao nhiêu?

# Định luật Zipf (Zipf's Law)

- Heaps' Law cho biết **kích thước từ vựng** trong các bộ sưu tập
- Ngoài ra, nghiên cứu **tần suất tương đối của các term** rất cần thiết
- Trong ngôn ngữ tự nhiên:
  - Có một số rất ít term xuất hiện với tần suất rất cao
  - Và có rất nhiều term xuất hiện với tần suất rất thấp
- **Định luật Zipf**
  - Term đứng thứ i theo mức độ phổ biến (tức là term phổ biến thứ i) có tần suất **tỷ lệ nghịch với i**
  - $cf_i \propto 1/i = K/i$
    - $K$ là hằng số chuẩn hóa
    - $cf_i$ là <u>collection frequency</u> — số lần xuất hiện của term $t_i$ trong toàn bộ bộ sưu tập

# Hệ quả của định luật Zipf

- Nếu từ phổ biến nhất (“the”) xuất hiện $cf_1$ lần,
  - thì từ phổ biến thứ hai (“of”) sẽ xuất hiện $cf_1$ / 2 lần,
  - từ phổ biến thứ ba (“and”) sẽ xuất hiện $cf_1$ / 3 lần, …
- Tương đương, ta có:
  - $cf_i = K/i$ , trong đó K là hằng số chuẩn hóa
  - $\log cf_i = \log K - \log i$
  - Có quan hệ tuyến tính giữa log $cf_i$ và log i



# Nén

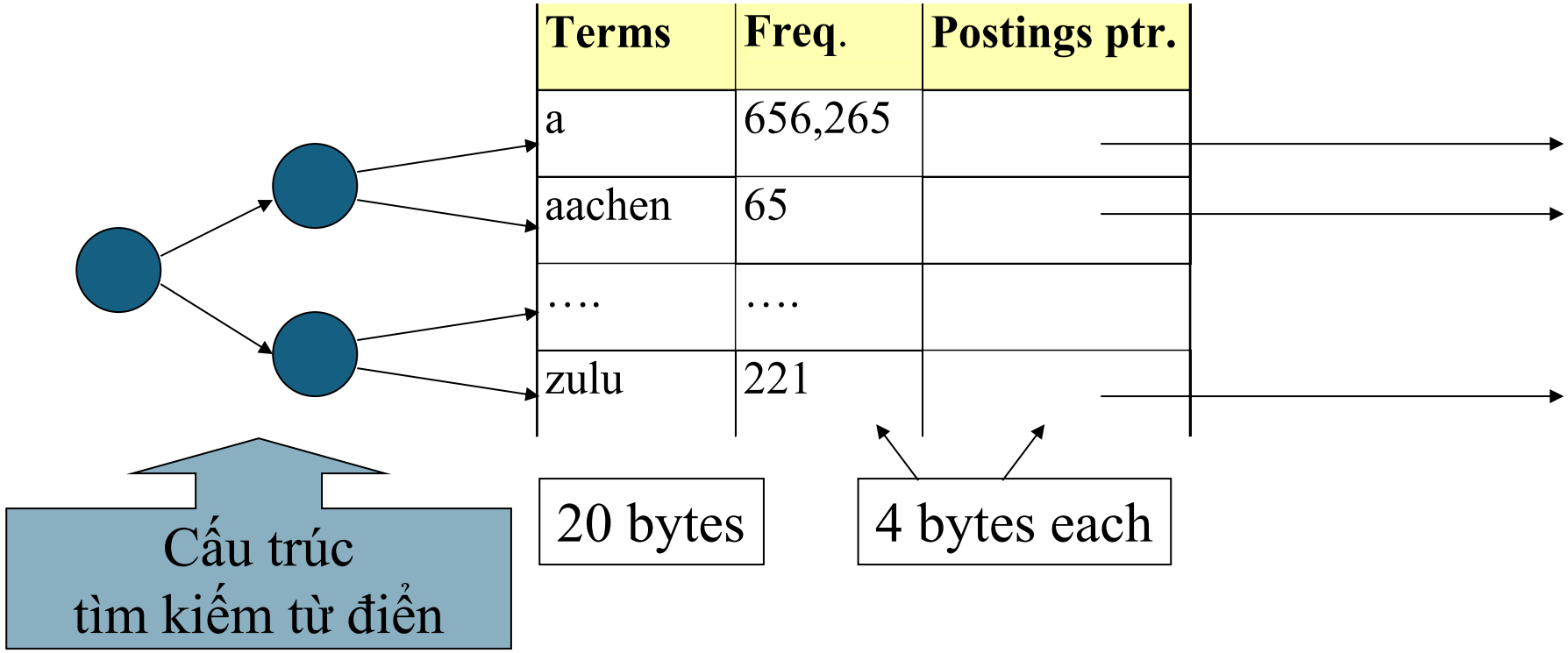
- Bây giờ, chúng ta sẽ xem xét việc nén không gian lưu trữ cho:
  - **Từ điển**
  - **Postings**
- Trong phần này, chúng ta **chỉ xét:**
  - Chỉ mục Boolean cơ bản
  - Không nghiên cứu chỉ mục có vị trí

# Vì sao cần nén từ điển?

- Quá trình tìm kiếm bắt đầu từ **từ điển**
- Chúng ta muốn lưu từ điển trong **bộ nhớ chính**
- Bộ nhớ phải **cạnh tranh** với các ứng dụng khác
- Các thiết bị nhúng / di động có **dung lượng bộ nhớ rất hạn chế**
- Ngay cả khi từ điển không được lưu hoàn toàn trong bộ nhớ, ta vẫn muốn nó nhỏ để **giảm thời gian khởi động hệ thống tìm kiếm**
- Vì vậy, **nén từ điển** là rất quan trọng

# Lưu trữ từ điển – phiên bản "ngây thơ"

- Mảng gồm các mục có **độ rộng cố định**
  - Khoảng ~400.000 terms; 28 byte/term ⇒ 11,2 MB

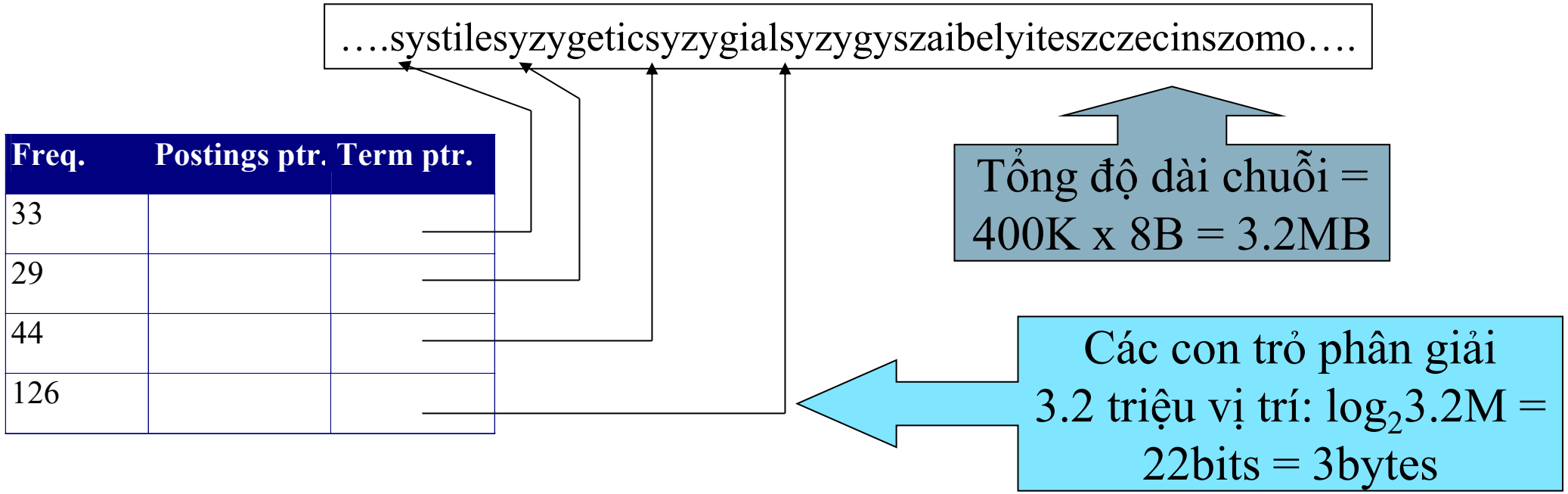


# Các terms có độ dài cố định gây lãng phí

- **Hầu hết các byte trong cột Term đều bị lãng phí** – chúng ta dành ra 20 byte cho cả những term chỉ có 1 chữ cái.
- **Và chúng ta vẫn không thể xử lý được** các từ cực dài như *supercalifragilisticexpialidocious* hay *hydrochlorofluorocarbons*.
- Tiếng Anh viết trung bình có khoảng 4.5 ký tự/từ.
  - **Tại sao con số này không nên được dùng để ước tính kích thước từ điển?**
- Một từ trung bình trong từ điển tiếng Anh: khoảng 8 ký tự.
  - **Làm thế nào để chúng ta sử dụng khoảng 8 ký tự cho mỗi mục từ trong từ điển?**
- Các từ ngắn chiếm ưu thế về số lượng token (tần suất xuất hiện) nhưng không chiếm ưu thế trong trung bình loại từ (type).

# Nén danh sách term: Từ điển dạng chuỗi

- Lưu trữ từ điển dưới dạng một chuỗi (dài) các ký tự:
  - Con trỏ trỏ đến **term tiếp theo** sẽ cho biết điểm kết thúc của **term hiện tại**.
  - Tiết kiệm được tới **60%** không gian từ điển.



# Không gian cho từ điển dạng chuỗi

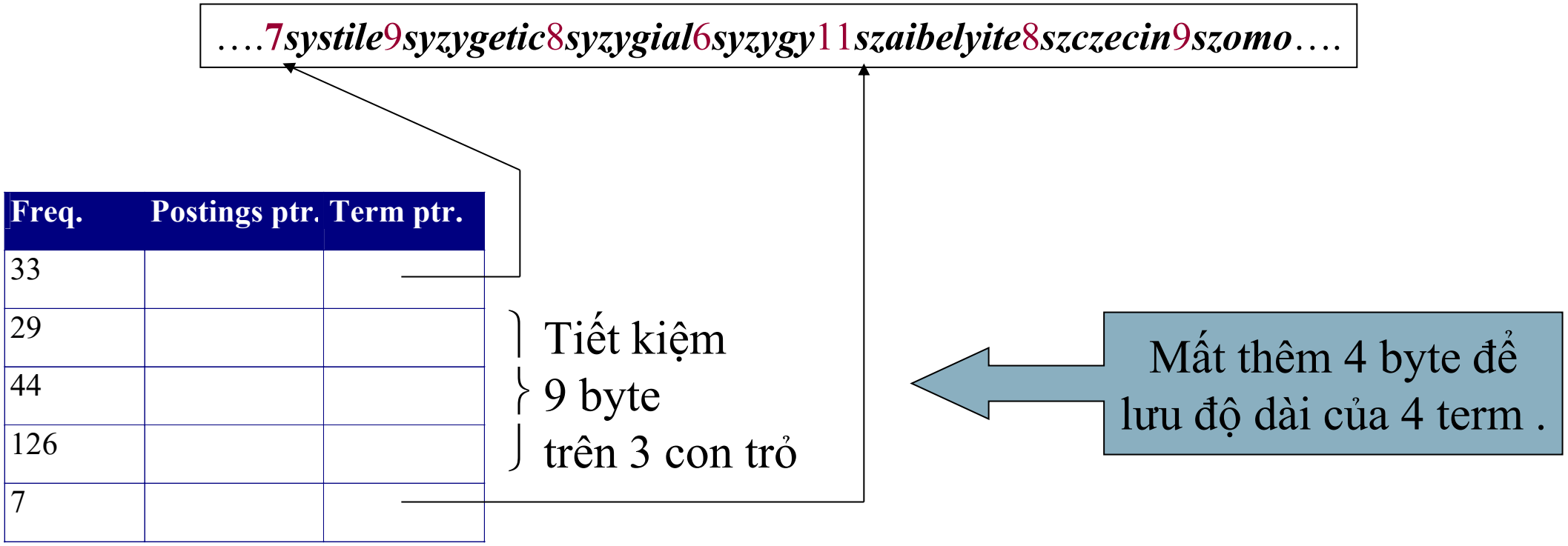
- **4 byte** mỗi term cho **tần suất**
- **4 byte** mỗi term cho **con trỏ tới Postings**
- **3 byte** mỗi term cho **con trỏ trỏ vào chuỗi ký tự**

Giờ trung bình 11 bytes/term, không phải 20.

- Trung bình **8 byte** mỗi term trong chuỗi ký tự
- Tính toán tổng cộng: **400K terms x 19 = 7.6 MB** (giảm so với 11.2 MB của phương pháp mảng độ dài cố định)

# Phân khối (Blocking)

- Lưu trữ con trỏ tới mỗi term thứ k trong chuỗi
  - Ví dụ: k = 4
- Cần lưu trữ thêm độ dài của mỗi term (thêm 1 byte)



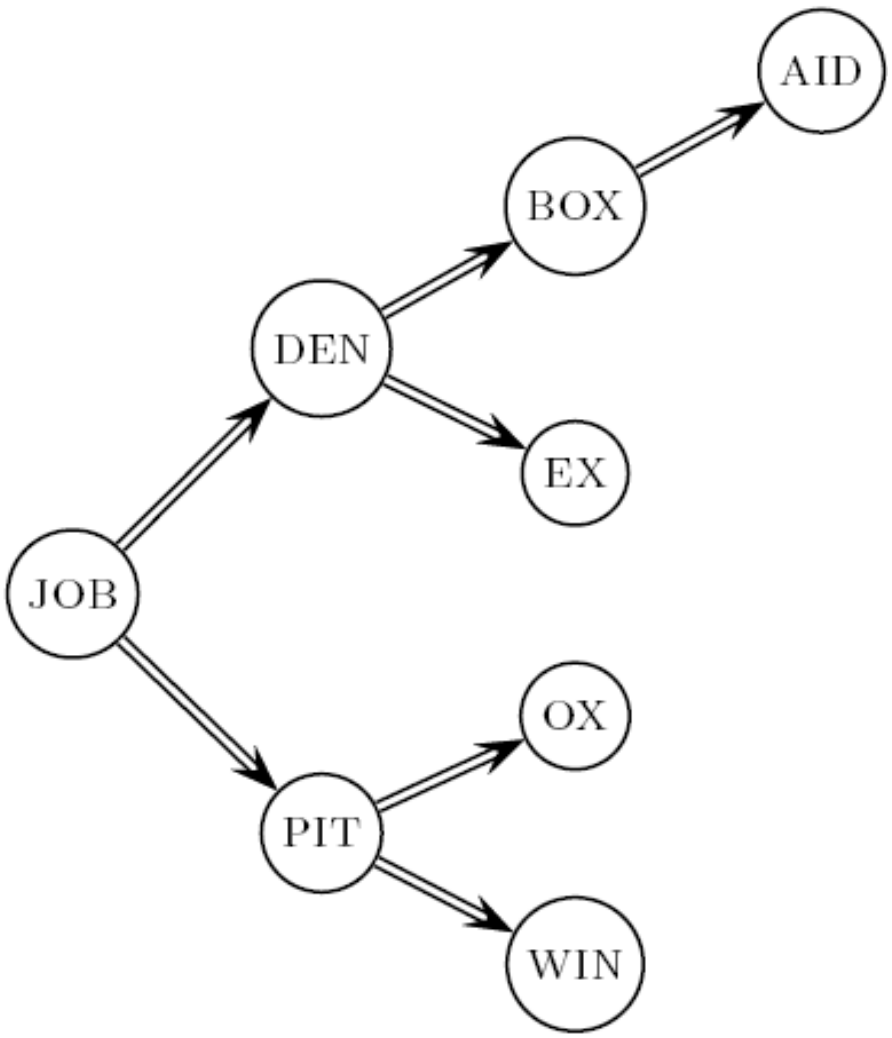
# Lợi ích từ phân khối

- Ví dụ cho kích thước khối k = 4
- **Trường hợp không phân khối:** sử dụng 3 byte cho mỗi con trỏ
  - Với 4 term, chi phí là: 3 x 4 = 12 byte.
- **Trường hợp có phân khối (k = 4):** Bây giờ chỉ sử dụng 3 + 4 = 7 byte.
  - **3 byte** cho 1 **con trỏ duy nhất** trỏ đến đầu khối
  - **4 byte** để lưu độ dài của 4 **term** trong khối đó (mỗi **term** tốn 1 byte để lưu độ dài)
  - Kỹ thuật này giúp cắt giảm thêm khoảng **0.5 MB**
  - Giảm tổng kích thước từ điển từ **7.6 MB** xuống còn **7.1 MB**
  - Có thể tiết kiệm nhiều hơn nữa nếu tăng giá trị k
- Tại sao không tăng k lên lớn hơn nữa?

# Tìm kiếm từ điển khi không phân khối

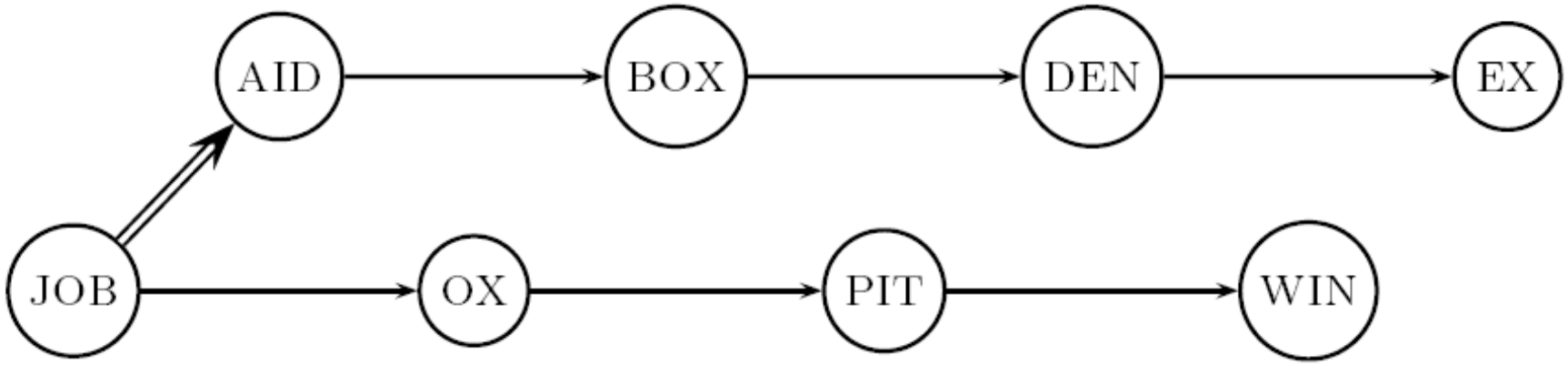
- **Giả định:** Mỗi **term** trong từ điển có khả năng được truy vấn ngang nhau (trong thực tế thì không hẳn như vậy!)
- **Số lần so sánh trung bình:** $(1+2\cdot2+4\cdot3+4)/8 \sim 2.6$
- Sử dụng cây tìm kiếm nhị phân giúp nhanh chóng xác định vị trí của một term

Nếu tần suất truy vấn các từ là không đồng đều nhưng đã biết trước (ví dụ từ "the" được tìm nhiều hơn từ "arachnocentric"), bạn sẽ cấu trúc cây tìm kiếm từ điển như thế nào để tối ưu hơn?



# Tìm kiếm từ điển khi có phân khối

- **Cơ chế:** Tìm kiếm nhị phân cho đến khi xác định được khối 4-term; sau đó thực hiện tìm kiếm tuyến tính qua các **term** trong khối đó
- **Ví dụ với khối bằng 4 (cây nhị phân):** Số lần so sánh trung bình = (1+2·2+2·3+2·4+5)/8 = 3 lần so sánh
- **Đánh đổi:** So với mức 2.6 lần khi không phân khối, tốc độ tìm kiếm ở đây chậm hơn một chút nhưng đổi lại dung lượng lưu trữ được tiết kiệm đáng kể



# Bài tập

- **Ước tính việc sử dụng không gian:** (và mức tiết kiệm so với con số 400.000 terms và 7.6 MB) khi áp dụng phân khối, cho các kích thước khối k = 4, 8 và 16.
- **Ước tính tác động đến hiệu suất tìm kiếm:** (và mức độ chậm đi so với trường hợp k = 1) khi áp dụng phân khối, cho các kích thước khối k = 4, 8 và 16.

# Mã hóa tiền tố

- Front-coding:
  - Các từ khi đã được sắp xếp thường có chung phần tiền tố dài – chúng ta chỉ lưu trữ những phần khác biệt.
  - (Kỹ thuật này áp dụng cho k - 1 từ còn lại trong một khối gồm k từ).

8*automata*8*automate*9*automatic*10*automation*

→8*automat*a*1◊*e*2◊*ic*3◊*ion*

**Phần mã hóa:** Ghi nhớ tiền tố chung là **automat**

**Độ dài bổ sung:** Chỉ lưu phần ký tự thừa ra sau đoạn **automat**

Cách tiếp cận này giống với các phương pháp nén chuỗi tổng quát.

# Tóm tắt nén từ điển RCV1

| Kỹ thuật | Kích thước tính bằng MB |
|---|---|
| Độ dài cố định | 11.2 |
| Từ điển dạng chuỗi với con trỏ tới từng term | 7.6 |
| + phân khối, k = 4 | 7.1 |
| + phân khối + Mã hóa tiền tố | 5.9 |

# Nén danh sách Postings

- **Tệp postings lớn hơn nhiều so với từ điển:** Thường lớn hơn ít nhất 10 lần, và đôi khi lên đến hơn 100 lần.
- **Yêu cầu then chốt:** Lưu trữ mỗi bản ghi posting một cách gọn nhẹ nhất có thể.
- **Định nghĩa:** Một bản ghi posting trong mục đích của chúng ta là một **docID** (mã định danh tài liệu).
- Ví dụ với tập dữ liệu RCV1 (800.000 tài liệu):
  - Chúng ta sẽ phải dùng **32 bit** cho mỗi docID nếu sử dụng kiểu số nguyên 4-byte thông thường.
  - Ngoài ra, chúng ta có thể dùng $\log_2 800{,}000 \approx 20$ bit cho mỗi docID.
- **Mục tiêu:** Sử dụng ít hơn nhiều so với con số 20 bit cho mỗi docID.

# Postings: Hai thế lực đối nghịch

- Một **term** như *arachnocentric* có thể chỉ xuất hiện trong 1 trên 1 triệu tài liệu – chúng ta lưu trữ **posting** này bằng $\log_2 1M \approx 20$ bits.
- Một **term** như *the* xuất hiện trong hầu hết mọi tài liệu, vì vậy việc dùng 20 bits cho mỗi **posting** ≈ 2MB là quá đắt đỏ.
- Trong trường hợp từ phổ biến như vậy, chúng ta ưu tiên sử dụng vector bitmap 0/1 (≈100KB)

# Mã hóa khoảng cách cho các mục trong postings file

- Chúng ta lưu trữ danh sách các tài liệu chứa một **term** theo thứ tự tăng dần của **docID**.
  - ***computer***: 33,47,154,159,202 …
- **Hệ quả:** Chỉ cần lưu trữ các **khoảng cách** giữa các **docID** là đủ.
  - 33,14,107,5,43 …
- **Hy vọng:** Hầu hết các khoảng cách có thể được mã hóa/lưu trữ với ít hơn rất nhiều so với 20 bits.
  - Điều này đặc biệt hiệu quả đối với các từ thông dụng (vì khoảng cách giữa các lần xuất hiện của chúng rất nhỏ).

# Ví dụ về ba mục nhập postings

| Term | Dạng mã hóa | Danh sách postings |
|---|---|---|
| **THE** | docIDs | ... 283042, 283043, 283044, 283045 ... |
| | **gaps** | **... 1, 1, 1, 1 ...** |
| **COMPUTER** | docIDs | ... 283047, 283154, 283159, 283202 ... |
| | **gaps** | **107, 5, 43 ...** |
| **ARACHNOCENTRIC** | docIDs | 252000, 500100 |
| | **gaps** | **252000, 248100** |

# Mã hóa độ dài biến đổi

- Mục tiêu:
  - Với ***arachnocentric***, chúng ta sẽ dùng khoảng **20 bits** cho mỗi mục nhập khoảng cách.
  - Với ***the***, chúng ta sẽ dùng khoảng **1 bit** cho mỗi mục nhập khoảng cách.
- Nếu khoảng cách trung bình của một **term** là *G* chúng ta muốn sử dụng khoảng $\log_2 G$ bits cho mỗi mục nhập.
- **Thách thức then chốt:** Mã hóa mọi số nguyên (khoảng cách) với số lượng bit xấp xỉ mức cần thiết cho số nguyên đó.
- **Giải pháp:** Điều này đòi hỏi một cơ chế **mã hóa độ dài biến đổi**.
- Các mã độ dài biến đổi đạt được điều này bằng cách sử dụng các mã ngắn cho các số nhỏ.

# Mã Unary

- Biểu diễn số *n* dưới dạng một chuỗi gồm *n* số 1 và kết thúc bằng một số 0.
- Mã Unary cho số 3 là 1110.
- Mã Unary cho số 40 là:

  1111111111111111111111111111111111111110

- Mã Unary cho số 80 là:

11111111111111111111111111111111111111111111111111111111111111111111111111110

- **Nhận xét:** Cách này có vẻ không tốt cho các số lớn, nhưng có thể sử dụng nó như một phần của giải pháp tổng thể.

# Mã Gamma

- Chúng ta có thể nén tốt hơn bằng các bộ mã ở cấp độ bit.
- Mã Gamma là bộ mã nổi tiếng nhất trong số này.
- Biểu diễn một khoảng cách *G* dưới dạng một cặp: **độ dài và phần bù**.
- **Phần bù (offset):** Là giá trị *G* dạng nhị phân, nhưng bị cắt bỏ bit đầu tiên (bit cao nhất)
  - Ví dụ: 13 → 1101 → 101
- **Độ dài (length):** Là độ dài của phần bù
  - Với số 13 (phần bù 101), độ dài là **3**.
- **Mã hóa độ dài bằng mã Unary:** Với độ dài là 3, mã Unary sẽ là **1110**.
- **Mã Gamma của 13** là sự kết hợp giữa **độ dài và phần bù: 1110101**.

# Ví dụ về mã Gamma

| Số | Độ dài | Phần bù | Mã Gamma | Tổng số bit |
|---|---|---|---|---|
| 1 | 0 | không có | 0 | 1 |
| 2 | 10 | 0 | 10,0 | 3 |
| 3 | 10 | 1 | 10,1 | 3 |
| 4 | 110 | 00 | 110,00 | 5 |
| 9 | 1110 | 001 | 1110,001 | 7 |
| 13 | 1110 | 101 | 1110,101 | 7 |
| 24 | 11110 | 1000 | 11110,1000 | 9 |
| 511 | 111111110 | 11111111 | 111111110,11111111 | 17 |
| 1025 | 11111111110 | 0000000001 | 11111111110,0000000001 | 21 |

# Các phép toán Bitwise

- Để thực hiện nén dữ liệu, chúng ta cần sử dụng các toán tử bitwise để tác động trực tiếp lên từng bit của số nguyên.
- Trong Python (và hầu hết các ngôn ngữ lập trình khác):
  - & : phép **AND** bitwise; | : phép **OR** bitwise; ^ : phép **XOR** bitwise; ~ : phép bù 1 (nghịch đảo bit).
  - << : dịch bit sang trái; >> : dịch bit sang phải.
  - *Lưu ý:* Python thiếu toán tử >>> (dịch phải chèn số 0 vào đầu) như trong Java.
- Các công thức:
  - **Trích xuất 7 bit:** a & 0x7f00 >> 8; nếu lấy các bit bậc cao thì thêm: & 0x7f.
  - **Kết hợp ba số 5-bit:** a | (b << 5) | (c << 10)
  - Việc sử dụng **bảng tra cứu** vì giải mã thủ công có thể giúp tốc độ xử lý nhanh hơn mà vẫn tốn ít bộ nhớ.

# Các đặc điểm của mã Gamma

- Một số $G$ được mã hóa bằng **$2\lfloor \log G \rfloor + 1$ bits.**
  - Độ dài của phần bù là **$\lfloor \log G \rfloor$ bits.**
  - Độ dài của phần độ dài là **$\lfloor \log G \rfloor + 1$ bits.**
- Tất cả các mã Gamma đều có số lượng bit là **số lẻ.**
- Mã này gần như **nằm trong phạm vi sai số** gấp 2 lần so với mức tối ưu lý tưởng $\log_2 G$
- Mã Gamma có tính chất **"uniquely prefix-decodable" (giải mã tiền tố duy nhất)**
  - Không cần dấu phẩy giữa các mã, máy tính vẫn biết khi nào một mã kết thúc và mã tiếp theo bắt đầu.
- Mã Gamma có thể được sử dụng cho **bất kỳ phân phối dữ liệu** nào.
- Mã Gamma **không phụ thuộc** vào tham số.

# Mã Gamma hiếm khi được sử dụng trong thực tế

- Máy tính có các **ranh giới từ cố định** – 8, 16, 32, 64 bits.
- Các thao tác xử lý vượt qua ranh giới từ sẽ **chậm hơn**.
- Việc nén và thao tác ở cấp độ từng bit (bit-level) có thể **quá chậm**.
- Tất cả các phương pháp thực tế hiện đại đều sử dụng các mã căn chỉnh **theo byte** hoặc **theo từ**.
  - **Mã hóa Variable Byte** là một sơ đồ nén nhanh hơn, khái niệm đơn giản hơn và có hiệu quả nén ở mức khá

# Mã Variable Byte (VB)

- **Mục tiêu:** Đối với một giá trị khoảng cách *G*, chúng ta muốn sử dụng số lượng byte gần nhất với mức tối thiểu cần thiết để chứa các bit của $\log_2 G$ bits
- **Cơ chế:** Bắt đầu với một byte để lưu trữ *G* và dành ra 1 bit trong đó làm **bit tiếp tục** *c*
- **Trường hợp số nhỏ:** Nếu $G \leq 127$, hãy mã hóa nhị phân nó vào 7 bit còn trống và đặt c = 1
- **Trường hợp số lớn:** Nếu không, hãy mã hóa 7 bit bậc thấp của *G*, sau đó sử dụng các byte bổ sung để mã hóa các bit bậc cao hơn bằng thuật toán tương tự
- **Kết thúc:** Cuối cùng, đặt bit tiếp tục của byte cuối cùng bằng 1 (c = 1) và đặt c = 0 đối với các byte khác.

# Ví dụ minh họa

| docIDs | 824 | 829 | 215406 |
|---|---|---|---|
| gaps | | 5 | 214577 |
| VB code | 00000110<br>10111000 | 10000101 | 00001101<br>00001100<br>10110001 |

Các **postings** được lưu dưới dạng chuỗi các byte nối tiếp nhau.

000001101011100010000101000011010000110010110001

Đặc điểm chính: Các postings được mã hóa VB có tính chất uniquely prefix-decodable

Đối với một khoảng cách nhỏ (5), mã VB vẫn phải sử dụng trọn vẹn một byte

# Nén RCV1

| Cấu trúc dữ liệu | Kích thước tính bằng MB |
|---|---|
| Từ điển, độ dài cố định | 11.2 |
| Từ điển, dùng con trỏ trỏ vào chuỗi | 7.6 |
| ... kết hợp Phân khối (with blocking), k = 4 | 7.1 |
| ... kết hợp Phân khối & Mã hóa tiền tố | 5.9 |
| **Toàn bộ tập dữ liệu** (văn bản, mã nguồn XML, v.v.) | 3,600.0 |
| **Toàn bộ tập dữ liệu** (chỉ văn bản thuần) | 960.0 |
| Ma trận xuất hiện Term-doc | 40,000.0 |
| **Postings**, chưa nén (dùng số nguyên 32-bit) | 400.0 |
| **Postings**, chưa nén (dùng 20 bits) | 250.0 |
| **Postings**, mã hóa Variable Byte | 116.0 |
| **Postings**, mã hóa Gamma | 101.0 |

# Các mã hóa đơn vị thay đổi khác

- Các mã **Variable Byte** được sử dụng bởi nhiều hệ thống thực tế.
- Đây là sự kết hợp hiệu quả giữa **mã hóa độ dài biến đổi** và khả năng tương thích với **căn chỉnh bộ nhớ máy tính**.
- Việc căn chỉnh theo byte gây **lãng phí không gian** nếu có nhiều khoảng cách nhỏ – điều thường xảy ra khi mã hóa khoảng cách
- Hầu hết các nghiên cứu hiện đại chủ yếu sử dụng các ý tưởng sau:
  - **Căn chỉnh theo từ:** Sử dụng các khối 32 hoặc 64 bit; giúp xử lý nhanh hơn nữa
  - Mã hóa nhiều khoảng cách **cùng một lúc**.
  - Thường giả định một **kích thước khoảng cách tối đa**, có thể kèm theo một mã thoát.

# Mã số nguyên biến độ dài theo nhóm

- **Group Variable Integer**
- **Được sử dụng bởi Google** vào khoảng đầu những năm 2000.
- **Cơ chế:** Mã hóa một nhóm 4 số nguyên trong các khối có kích thước từ 5–17 byte.
- **Byte đầu tiên:** Chứa bốn trường độ dài 2-bit mô tả kích thước của 4 số tiếp theo.
  - $\boxed{L_1 \mid L_2 \mid L_3 \mid L_4}$ , $L_j \in \{1,2,3,4\}$
  - **Các byte tiếp theo:** 4 đến 16 byte tiếp theo sẽ chứa nội dung của 4 số đó.
    - Mỗi số có thể sử dụng 8/16/24/32 bit.
    - Khoảng cách tối đa lên tới ~4 tỷ.
- **Hiệu năng:** Được đánh giá là nhanh gấp đôi so với mã hóa **Variable Byte** truyền thống.
  - Việc giải mã các **khoảng cách** đơn giản hơn nhiều vì không cần các thao tác mặt nạ bit phức tạp; byte đầu tiên có thể được giải mã nhanh chóng bằng bảng tra cứu.

# Simple-9 [Anh & Moffat, 2004]

- **Ý tưởng:** Đóng gói tối đa số nguyên có thể vào một **word 32-bit**.
- **Cấu trúc:**
  - **4 bit đầu tiên:** Dùng để biểu thị "layout" (cách sắp xếp). Có 9 layout khả thi (đó là lý do gọi là Simple-9).
  - **28 bit còn lại:** Dùng để lưu trữ dữ liệu thực tế - các khoảng cách.
- Các layout khả thi:
  1. 28 số nguyên, mỗi số 1 bit.
  2. 14 số nguyên, mỗi số 2 bit.
  3. 9 số nguyên, mỗi số 3 bit (dư 1 bit).
  4. 7 số nguyên, mỗi số 4 bit.
  5. 5 số nguyên, mỗi số 5 bit (dư 3 bit).
  6. 4 số nguyên, mỗi số 7 bit.
  7. 3 số nguyên, mỗi số 9 bit (dư 1 bit).
  8. 2 số nguyên, mỗi số 14 bit.
  9. 1 số nguyên, mỗi số 28 bit.

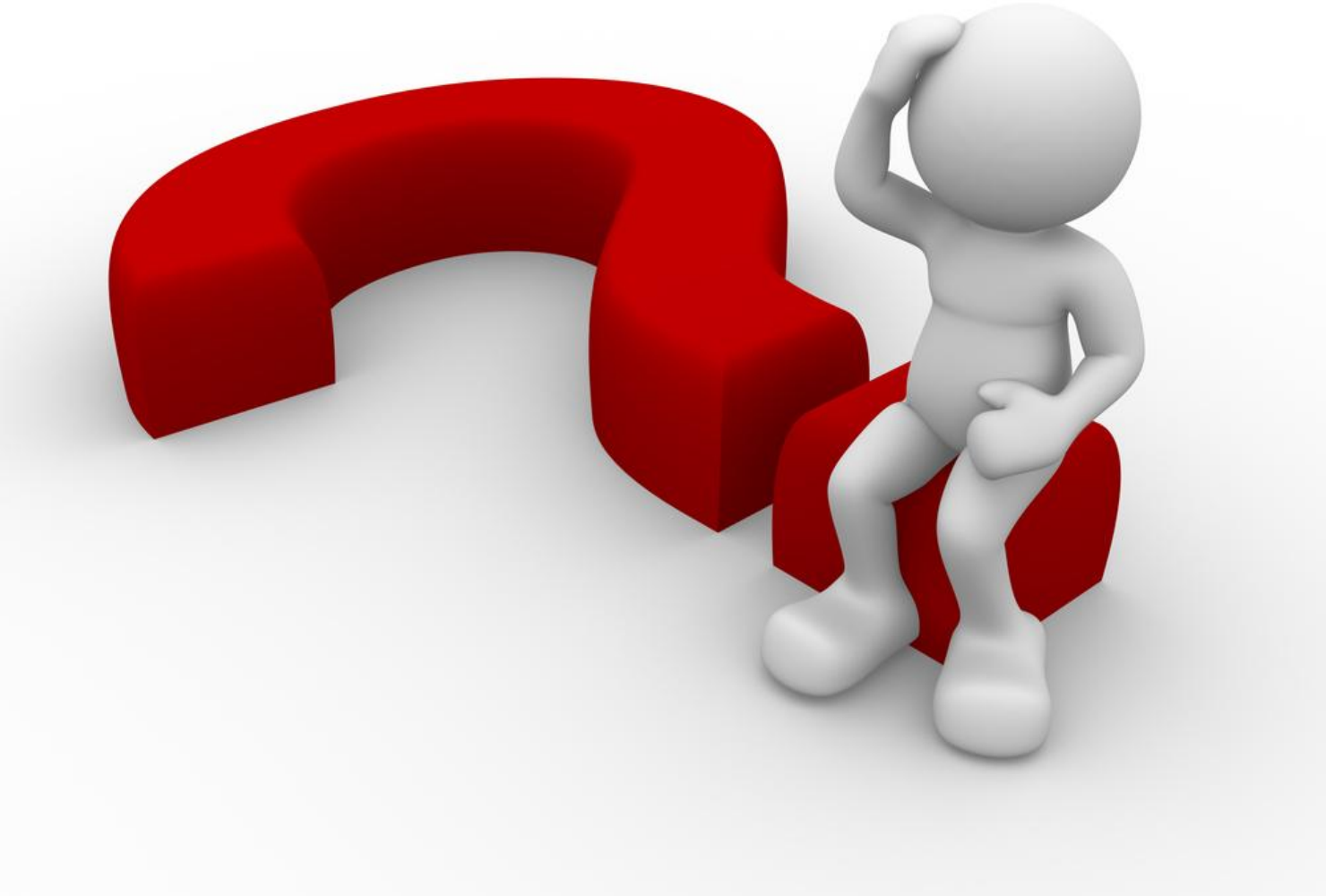
| Layout (4 bits) | n numbers of b bits each<br>$n * b \leq 28$ |
|---|---|

- **Simple16** là một biến thể với **5 cấu hình bổ sung**
- Có thể được **giải mã hiệu quả** bằng bộ giải mã viết thủ công, sử dụng **mặt nạ bit**
- **Họ Extended Simple** – ý tưởng này có thể áp dụng cho **từ 64-bit (64-bit words)**, v.v.

# Tổng kết nén danh sách Postings

| Phương pháp mã hóa | Kích thước RCV1 | Tốc độ giải mã | Đặc điểm nổi bật |
|---|---|---|---|
| **Số nguyên 32-bit** | 400 MB | Rất nhanh | Lãng phí bộ nhớ khủng khiếp. |
| **Mã Gamma** | 101 MB | Chậm | Tỷ lệ nén tốt nhất nhưng xử lý ở cấp độ bit nên CPU rất vất vả. |
| **Mã Variable Byte** | 116 MB | Nhanh | Cân bằng tốt, dễ cài đặt, cực kỳ phổ biến. |
| **Simple-9 / Simple-16** | ~110 MB | Rất nhanh | Tối ưu cho kiến trúc CPU 32/64-bit, giải mã hàng loạt số cùng lúc. |
| **Group VarInt** | ~115 MB | Cực nhanh | Cách tiếp cận của Google, giảm thiểu các lệnh rẽ nhánh (branching) giúp CPU chạy mượt nhất. |

# Tóm tắt nén chỉ mục

- Chúng ta có thể xây dựng một **chỉ mục** cho truy hồi Boolean có **hiệu năng rất cao** và **rất tiết kiệm không gian lưu trữ**
- Kích thước chỉ khoảng **4% tổng kích thước của bộ sưu tập**
- Chỉ khoảng **10–15% tổng kích thước phần văn bản trong bộ sưu tập**
- Chúng ta đã **bỏ qua thông tin vị trí**
- Do đó, trong thực tế khi sử dụng **chỉ mục vị trí**, mức tiết kiệm không gian sẽ thấp hơn
  - Tuy nhiên, **các kỹ thuật nén được sử dụng về cơ bản là tương tự nhau**.

# Hỏi đáp